Product Requirements Document

VolunteerMatch AI — Smart Resource Allocation

H2S × Google for Developers: Build with AI 2026

---

1. Project Overview

Project Name: VolunteerMatch AI
Track: Smart Resource Allocation
Challenge: Data-Driven Volunteer Coordination for Social Impact
Hackathon: Hack2Skill × Google for Developers — Solution Challenge 2026

Problem Statement

Local social groups and NGOs collect critical community needs data through paper surveys and field reports. This data is siloed and scattered, making it impossible to see where help is needed most or quickly connect available volunteers to urgent tasks.

Our Solution

A web-based AI platform that lets NGO admins input community needs and available volunteers. The system uses Gemini AI to intelligently match volunteers to tasks based on skills, location, and urgency — in real time.

USP (Unique Selling Point)

Unlike manual spreadsheets or generic CRMs, VolunteerMatch AI uses Google's Gemini model to reason over unstructured volunteer profiles and community needs, producing prioritized, explainable matches with natural language justifications. No data science expertise required.

---

2. Hackathon Compliance Checklist

Requirement	How We Meet It
Cloud deployment	Deployed on Firebase Hosting / Vercel
Google AI model	Gemini API for volunteer-need matching
Live MVP link	Deployed public URL
GitHub public repository	Open source repo with clean README
Demo video (3 min)	Screen-recorded walkthrough

Requirement	How We Meet It
Project deck (PPT template)	Filled per official template

3. Evaluation Criteria Alignment

Criterion	Weight	Our Strategy
Technical Merit	40%	Gemini API integration, clean codebase, Firebase deployment, scalable data model
Alignment With Cause	25%	Directly addresses NGO resource gaps — a real, measurable social impact problem
Innovation & Creativity	25%	AI-powered matching with natural language explanations; no comparable free tool exists
User Experience	10%	Clean dashboard UI, mobile-responsive, zero learning curve for NGO staff

4. Core Features (MVP Scope)

Feature 1 — Community Needs Board

- NGO admin can add a "Need" with: title, description, required skills, location, and urgency level (Low / Medium / Critical)
- Needs displayed as cards on a dashboard, color-coded by urgency
- Admin can mark a need as "Fulfilled" or "In Progress"

Feature 2 — Volunteer Registry

- Admin can add volunteers with: name, skills (multi-select tags), availability, and area/zone
- Volunteer profiles stored and browsable

Feature 3 — AI Matching Engine (Gemini)

- Admin clicks "Find Best Match" on any Need card
- App sends the Need + list of available volunteers to Gemini API
- Gemini returns a ranked list of top 3 volunteers with a natural language explanation for each match

- Example output: *"Priya is recommended because she has medical training, is available this weekend, and operates in the same zone as the reported need."*

Feature 4 — Match Dashboard

- Central view showing: Open Needs, Matched Pairs, Fulfilled Tasks
- Simple stats: Total Volunteers, Open Needs, Matches Made This Week
- Filter by urgency, skill type, or zone

Feature 5 — One-Click Notifications (Stretch Goal)

- When a match is confirmed, send a simple email notification to the volunteer via EmailJS or Firebase Functions

---

5. Technical Architecture

Tech Stack (All Vibecode-Friendly)

Layer	Technology	Why
Frontend	React (via Bolt.new or Lovable)	Fast vibecoding, clean UI
Styling	Tailwind CSS	Beautiful out of the box
Backend/DB	Firebase Firestore	No-backend setup, real-time data
Auth	Firebase Authentication	Google sign-in in 10 minutes
AI	Google Gemini API (gemini-1.5-flash)	Required by hackathon; free tier available
Hosting	Firebase Hosting	Free, instant deployment, satisfies cloud req

Data Model

```
needs/
  {needId}
    title: string
    description: string
    skills_required: string[]
    urgency: "low" | "medium" | "critical"
    location: string
    status: "open" | "matched" | "fulfilled"
    created_at: timestamp
```

```
volunteers/
  {volunteerId}
    name: string
    skills: string[]
    availability: string
    zone: string
    is_available: boolean

matches/
  {matchId}
    need_id: string
    volunteer_id: string
    ai_justification: string
    confirmed: boolean
    created_at: timestamp
```

Gemini API Call (Core Logic)

```javascript
const prompt = `
You are a volunteer coordinator AI.
Given this community need:
${JSON.stringify(need)}

And these available volunteers:
${JSON.stringify(volunteers)}

Return a JSON array of the top 3 best volunteer matches, ranked by fit.
Each item should have: volunteer_id, volunteer_name, score (1-10), reason (1 sentenc
Return ONLY valid JSON. No markdown, no explanation outside the array.
`;
```

---

6. User Flow

```
NGO Admin logs in (Google Auth)
        ↓
Dashboard — sees open needs + volunteer count
        ↓
Adds a new Community Need (form)
        ↓
Clicks "Find AI Match" on a Need card
        ↓
Gemini API returns top 3 volunteers with reasons
```

```
        ↓
Admin confirms a match → status updates to "Matched"
        ↓
(Optional) Volunteer receives email notification
        ↓
Admin marks need as "Fulfilled" when done
```

---

7. Wireframe Descriptions (for PPT Slide 7)

Screen 1 — Dashboard
Top bar: App name + Google Auth avatar. Three stat cards: Open Needs / Active Volunteers / Matches This Week. Below: two columns — Needs Board (left) and Recent Matches (right).

Screen 2 — Add Need Modal
Form fields: Title, Description, Skills Required (tag input), Urgency (radio: Low/Medium/Critical), Location/Zone. Submit button.

Screen 3 — AI Match Results Panel
Triggered from a Need card. Shows 3 volunteer cards side by side. Each card: Volunteer name, skill tags, match score (e.g., 9/10), and Gemini's reason in italic text. "Confirm Match" button on each card.

Screen 4 — Volunteer Registry
Table/card grid of all volunteers. Each card: name, skill tags, zone, availability badge (green = available). "Add Volunteer" button top right.

---

8. Build Plan — Day by Day

Day 1 — Setup & Skeleton

- Create React app via Bolt.new or Lovable with prompt: *"Build a volunteer coordination dashboard with a needs board and volunteer registry using Tailwind CSS"*
- Set up Firebase project: enable Firestore + Authentication
- Connect app to Firebase
- Deploy first empty version to Firebase Hosting — **get live URL early**

Day 2 — Core UI

- Build the Needs Board: cards with urgency color-coding
- Build Add Need form (modal)
- Build Volunteer Registry page with Add Volunteer form
- All data saving to Firestore

Day 3 — Gemini AI Integration

- Get Gemini API key from Google AI Studio (aistudio.google.com) — free
- Write the matching function (send need + volunteers to Gemini, parse JSON response)
- Build the Match Results panel UI
- Test with 5-10 sample volunteers and needs

Day 4 — Polish & Dashboard Stats

- Add stats cards (open needs count, volunteer count, matches count)
- Add status updates (open → matched → fulfilled flow)
- Color-code urgency badges
- Make it mobile-responsive

Day 5 — Testing & Demo Prep

- Add 10-15 realistic dummy data entries (real NGO scenario — e.g., flood relief)
- Record 3-minute demo video following this script:
    1. Show the problem (30 sec)
    2. Log in and tour the dashboard (30 sec)
    3. Add a new urgent need (30 sec)
    4. Hit "Find AI Match" — show Gemini's ranked results with reasons (60 sec)
    5. Confirm a match, show status update (30 sec)

Day 6 — PPT Submission

Fill the official PPT template slide by slide:

Slide	Content
2 — Team Details	Your team name, leader, problem statement
3 — Solution Brief	"AI-powered volunteer matching platform for NGOs using Gemini"
4 — Opportunities	USP: explainable AI matches, no existing free tool, works with zero technical NGO staff
5 — Features	5 features listed above
6 — Process Flow	Use the User Flow diagram from Section 6 of this PRD
7 — Wireframes	Screenshots of your actual built app
8 — Architecture	Frontend → Firebase → Gemini API diagram
9 — Technologies	React, Tailwind, Firebase, Gemini API
11 — MVP Snapshots	Screenshots of the live app
13 — Links	GitHub repo, demo video, live URL

Day 7 — Buffer

- Fix any bugs
- Improve UI polish
- Re-record demo if needed
- Double-check all submission links work

---

9. Submission Requirements Checklist

- [ ] Live MVP URL (Firebase Hosting)
- [ ] GitHub public repository with README
- [ ] Demo video (max 3 minutes, uploaded to YouTube or Google Drive)
- [ ] Project deck (filled PPT template)
- [ ] Problem statement clearly defined
- [ ] Solution overview written
- [ ] Google AI (Gemini) integrated and demonstrable

---

10. Risks & Mitigations

Risk	Mitigation
Gemini API returns non-JSON	Add a try/catch + fallback prompt asking it to retry in JSON only
Firebase free tier limits	Well within limits for a demo; Firestore allows 50k reads/day free
Real-time sync looks broken in demo	Pre-load with dummy data; demo in one browser window
Judges ask "does this scale?"	Answer: Firestore auto-scales; Gemini API handles concurrent requests
Looks too simple	Focus demo on the *quality of Gemini's natural language justifications* — that's the wow factor

---

11. Demo Script Talking Points

1. **Hook:** "Every day, NGOs lose hours trying to manually match volunteers to needs using WhatsApp groups and spreadsheets. Critical help arrives too late."
2. **Solution:** "VolunteerMatch AI centralizes this and uses Google's Gemini to instantly surface the right volunteer for every need — with a reason you can trust."
3. **Show the AI:** The most powerful moment is when Gemini explains *why* it matched someone. Let that response speak for itself.
4. **Impact:** "This tool means an NGO coordinator with zero tech background can coordinate 10x more volunteers in the same time."

PRD Version 1.0 — Prepared for H2S × Google Solution Challenge 2026